Trabajo Práctico Integrador

Calidad de Software aplicada al Trabajo Final de Grado: **VoluntariAR**

Integrantes	Santiago Planas Montilla — legajo 2111770
	Santiago Francisco Antonio Ruibal — legajo 2009644
Docente	Pablo Alejandro Virgolini

Nivel declarado	**Nivel 3 — prototipo avanzado**
Fecha de presentación	26 de octubre de 2026
Repositorio	github.com/SantiagoPlanasM/voluntariar-PF

Todas las cifras de este informe provienen de ejecuciones del propio proyecto (GitHub Actions, SonarCloud, k6 y el registro de defectos en GitHub Issues), con la fuente indicada en cada caso. Corte de datos: 18/10/2026. Último cambio de código: `d7efe16` (CI #29 y Deploy #22: 10 de las 14 issues aceptadas de SonarCloud, arregladas en el código); los commits posteriores son solo de documentación.

Índice

1. Resumen y nivel declarado
2. Metas de calidad: cumplidas y no cumplidas
3. Los ejes del trabajo (8 temas + documentación)
4. Los temas de la materia, uno por uno
5. Validaciones de entrada: el código y sus pruebas
6. Evolución de las métricas
7. Análisis crítico
8. Conclusiones
9. Anexo: entregables, cómo verificar cada número y uso de IA

1. Resumen y nivel declarado

VoluntariAR es una red social de voluntariado que conecta voluntarios, ONG y empresas. Backend en Node 22 + Express con SQL a mano (SQLite en desarrollo y pruebas, Postgres en producción); frontend en React 18 + Vite; desplegado en Render (plan gratuito). La **integración externa** evaluada es **Mercado Pago** (donaciones con Checkout Pro, en modo prueba). Desde el 18/10 funciona como **marketplace**: cada ONG cobra en su propia cuenta de vendedor y la plataforma no cobra comisión. También usa Cloudinary (archivos) y Resend (emails).

Se declaró el **nivel 3 (prototipo avanzado)**, porque el producto estaba funcionalmente terminado y desplegado. El trabajo agregó lo que faltaba para que la calidad se apoye en datos: **pipeline de CI/CD** con quality gate, **SonarCloud** como herramienta nueva, **deploy automático con smoke y E2E contra Mercado Pago**, **pruebas de stress y esfuerzo sostenido**, plan de pruebas versionado y **registro de defectos en GitHub Issues** con trazabilidad.

> **En números:** 907 tests de backend en SQLite y 909 en Postgres (UTC y zona horaria de Córdoba), 904 de frontend, todos en verde (CI #29). Cobertura del backend 93,22 % de líneas; del frontend, 90,42 % de líneas y más del 80 % en sentencias, ramas y funciones. SonarCloud: issues abiertas **157 → 0**, bugs **39 → 0**, vulnerabilidades **18 → 0** abiertas; aceptadas con justificación **14 → 4**; cobertura del código nuevo 93,0 %. 2 ejecuciones fallidas reales del CI, ambas corregidas en commits posteriores. 18 defectos registrados con pasos, esperado, obtenido, commit y prueba de regresión. 61 commits durante el TP.

2. Metas de calidad: cumplidas y no cumplidas

Cada aspecto de la tabla del enunciado, contra lo exigido para el nivel 3.

Aspecto	Exigido	Logrado (evidencia)	Estado
Pipeline	Compilación, pruebas, análisis estático, cobertura, quality gate y reporte automático	`ci.yml`: build y tipos del frontend, tests (SQLite y Postgres × UTC y Córdoba), cobertura con umbral, SonarCloud + gate, `npm audit`; resumen y artefactos por run. CI #29 en verde (25 de 27 runs terminados; 3 cancelados por un push posterior)	**Cumplida**
Quality gate	Bloquea ante bugs críticos, duplicación > 5 % y cobertura del código nuevo < 80 %	`sonar-gate.js` (también bloquea vulnerabilidades críticas). Bloqueó de verdad en el **CI #7** (2 bugs críticos). Hoy: 0 / 0 / 1,9 % / 93,0 %	**Cumplida**

Aspecto	Exigido	Logrado (evidencia)	Estado
Cobertura del código nuevo	80 %	**93,0 %** (SonarCloud, análisis de `d7efe16`)	**Cumplida**
Plan de pruebas y riesgos	Versionado, criterios de entrada y salida, riesgos P × I, trazabilidad requisito → caso → defecto	`PLAN_PRUEBAS.md` v0.9 (9 versiones en git), 10 riesgos priorizados, criterios, matriz ISO 25010; `REGISTRO_DEFECTOS.md`	**Cumplida**
Defectos	Con pasos, esperado y obtenido, vinculados a su caso, en Jira u otra	18 Issues en GitHub (#2–#19) con plantilla, labels, caso, commit del arreglo y prueba de regresión	**Cumplida**
Herramienta nueva	Proceso de 5 pasos	SonarCloud: requisitos, mercado (4 alternativas), prueba de concepto (PR #1), matriz (92 contra 84 de Codacy), despliegue incremental	**Cumplida**
Cobertura del código existente	80 % (nivel 3)	Backend (CI #29): líneas 93,22 %, ramas 85,40 %, funciones 90,21 %. Frontend (CI #29): líneas **90,42 %**, sentencias **86,79 %**, ramas **81,57 %** y funciones **82,66 %**, con umbral bloqueante en las cuatro. En el CI #15 ramas y funciones no llegaban (74,08 % y 67,99 %)	**Cumplida**
Smoke test	Ejecutado por el pipeline tras el despliegue	`deploy.yml` → `post-deploy.js smoke` después de cada deploy (smoke en verde en los 15 deploys que llegaron a desplegar; el #14 se cortó antes, en la guarda de variables)	**Cumplida**
Integración externa	Flujo E2E automatizado en cada despliegue	En cada deploy: login → el proyecto de prueba existe, está activo y su ONG tiene Mercado Pago vinculado → donación → **preferencia creada en Mercado Pago real, con el token de la ONG** → estado pendiente. **Sin el pago**: no es automatizable en modo prueba (403 de CloudFront fuera de un navegador). El pago completo, verificado a mano 4 veces; la última, con el marketplace, acreditó en la cuenta de la ONG	**Cumplida, con un límite**
Stress y esfuerzo sostenido	Stress ejecutado, con resultados y umbrales	Workflow "Carga" (k6): stress p95 3,06 ms (umbral 500), sostenido p95 3,17 ms (umbral 300), producción p95 418,90 ms (umbral 2000); 0 % de errores	**Cumplida**

3. Los ejes del trabajo

3.1 Introducción a la calidad: objetivos y contingencia

13 objetivos medibles agrupados por **usuario** (rendimiento, deploys verificados, seguridad, accesibilidad), **equipo** (cero regresiones, cobertura, regresión por defecto, tiempo de recuperación) y **organización** (costo $0, dependencias sin vulnerabilidades, defectos escapados, continuidad de la demo). Cada uno lleva la práctica que lo sostiene y su valor medido (`PLAN_PRUEBAS.md §2.1`). El **plan de contingencia** por dependencia (`§3`) documenta el comportamiento real del código: Mercado Pago y Cloudinary responden 503 sin configuración y cortan a los 20 s con 502; Resend se omite sin romper; ante Render dormido, se despierta con `/health` o se usa el plan B local.

3.2 Fundamentos del testing

- **Arrange–Act–Assert y criterio previo:** los tests nuevos marcan los tres bloques y llevan el criterio de aceptación en el encabezado, escrito antes del código (por ejemplo, `sonar-gate.test.js`, `donation-amount.test.js`).
- **Defecto real con contraste rojo/verde:** B7, el filtro de palabras que buscaba subcadenas. El mismo test de regresión corrido en worktrees aparte: sobre el commit **anterior al arreglo**, 2/2 en rojo ("computadora" → 400, "imbécil" → 201); **sobre el arreglo**, 2/2 en verde. Evidencia: `docs/calidad/evidencia/b7-*.txt`.
- **Los siete principios**, con un ejemplo del proyecto cada uno (`PLAN_PRUEBAS.md §9`). El más visible: "*las pruebas muestran la presencia de defectos, no su ausencia*". Con la suite en verde, 4 defectos aparecieron recién en producción.
- **Mutation testing** sobre todo el código nuevo, con corrida de control. Encontró huecos reales en los tests: por ejemplo, el mínimo de 8 caracteres de la contraseña no tenía prueba de borde (D14).

3.3 Modelos de calidad y niveles de prueba

Matriz ISO/IEC 25010 (8 características) con evidencia y brecha de cada una (`PLAN_PRUEBAS.md §5`). **Top 3: seguridad** (CV privado, borrado de cuenta, Ley 25.326), **fiabilidad** (donaciones y cupos sin doble acreditación) y **compatibilidad** (la diferencia SQLite/Postgres fue la fuente de defectos más frecuente: B1, B2, B8, A6). **Mapa de niveles:** unitario, integración de API contra base real, componentes, sistema (smoke), integración externa (E2E), no funcional (carga) y aceptación (guion de la demo). **Despliegue verificado en cada instancia:** local y CI (suites en cada push) y producción (smoke + E2E en cada deploy; `/health` informa el commit desplegado).

3.4 Métricas y análisis estático

SonarCloud en el pipeline (`METRICAS.md`; análisis de `d7efe16`): 17.594 líneas de código; duplicación 1,9 %; complejidad ciclomática 4.245 y cognitiva 2.856 (la más alta: `routes/projects.js`, 333); deuda 0 min; calificaciones A/A/A. **Métrica de proceso:** tasa de éxito del pipeline (CI 92,6 %, Deploy 80 %) y tiempo hasta volver a verde (5,1 y 30 min en los fallidos del CI; 18 min en el Deploy #14). **Métrica de proyecto:** densidad de pruebas, con 145,5 tests/KLOC en el backend y 75,9 en el frontend (CI #19; antes, 60,6). **Mejora medida:** la evolución de SonarCloud de la sección 6.

3.5 Pruebas dinámicas

`DISENO_CASOS.md`:

- **Monto de la donación:** 5 clases de equivalencia y valores límite 99/100/101 y 499.999/500.000/500.001.
- **Contraseña:** bordes de 7/8 caracteres y de 72/73 bytes, también con ñ.
- **Acceso al CV:** tabla de decisión con 10 reglas, cada una con su test. Se agregó la que faltaba.
- **Cobertura de sentencia y de decisión** de `amountError`: 100 % de las dos con todas las clases. Con un solo caso válido se ejecutan 6/6 líneas pero solo 1 de 4 ramas: es el ejemplo de por qué una no implica la otra.

3.6 Aseguramiento de la calidad

Plan de pruebas versionado (v0.1 → v0.9) con criterios de entrada, salida y suspensión; **10 riesgos** priorizados por probabilidad × impacto (los más altos: Render dormido en la demo = 9; consultas que fallan solo en Postgres y la carga que tira el servicio gratuito = 6). **QA** (las 24 reglas de `AI_RULES.md`, tests primero, revisiones de código, gate) distinguido de **QC** (las suites, el smoke, la carga, la verificación en producción). **Plantillas de caso y de defecto** (`PLAN_PRUEBAS.md §10` y `.github/ISSUE_TEMPLATE/defecto.md`) y **trazabilidad** requisito → caso → defecto → Issue (`REGISTRO_DEFECTOS.md`).

3.7 Automatización del testing

- **Pirámide:** la base es ancha en integración de API contra base real (donde aparecieron casi todos los defectos), con unitarias, componentes, smoke, E2E y carga encima.
- **Pruebas de API**: 62 archivos de integración. **Integración con la plataforma externa**: contrato y dobles, prueba real a mano y E2E en cada deploy (`INTEGRACION_MERCADOPAGO.md`). El marketplace suma pruebas de la cuenta de cada ONG: el cifrado de sus tokens, el `state` de un solo uso, la renovación concurrente del token y que un pago solo se acredite si se leyó con la cuenta que cobró.
- **CI/CD:** `ci.yml` → `deploy.yml` (deploy hooks de Render, espera del commit en `/health`, smoke y E2E), con el auto-deploy de Render apagado.
- **Una prueba de regresión por cada defecto corregido:** las 18 de `REGISTRO_DEFECTOS.md`, todas en el CI.

3.8 Herramientas del ecosistema

Mapa de herramientas por propósito (`HERRAMIENTA_NUEVA.md`): node:test, Vitest + Testing Library, TypeScript 7, Postgres en Docker o como servicio, GitHub Actions, SonarCloud, k6, `npm audit`, GitHub Issues + `gh`, Chrome. **Herramienta nueva:** SonarCloud, con el proceso de 5 pasos y la matriz con pesos (SonarCloud 92, Codacy 84, Qlty 72, ESLint + jscpd 65). **Registro de defectos:** GitHub Issues. **Stress y esfuerzo sostenido:** k6 2.3.0 en el workflow "Carga".

3.9 Documentación (transversal)

Cada cambio dejó su sección en `PROJECT_ANALYSIS.md` (§183–§225), con qué se hizo, verificación, **errores propios** y **no verificado**. Los documentos del TP están en `docs/calidad/` (anexo).

4. Los temas de la materia, uno por uno

Cada tema del programa, con el nombre que tiene en la materia, dónde se aplicó y la evidencia. Cuando un tema no se aplicó o se aplicó a medias, se dice.

4.1 Fundamentos de calidad y testing

Calidad de software. La medimos en cuatro dimensiones:

- **Conformidad:** 12 requisitos funcionales trazados a sus tests (`PLAN_PRUEBAS.md §10`).
- **Eficiencia de desempeño:** p95 con umbrales fijados antes de correr k6 (sección 6.4).
- **Mantenibilidad:** SonarCloud A, deuda 0 min y duplicación 1,9 %.
- **Satisfacción del usuario:** no se midió con usuarios externos (ver "calidad en uso", 4.2).

Concepto	Definición	En VoluntariAR (defecto B7, y el ReDoS)
Error	Acción humana equivocada	Decidir que el filtro busque *subcadenas* sin normalizar acentos. En el email: escribir un regex con clases que se superponen
Defecto	La falla incorporada al código	El `includes` sobre el texto en el filtro de palabras. El `EMAIL_RE` viejo, con backtracking cuadrático
Fallo	La manifestación visible al ejecutar	"Traigan la computadora" rechazado con 400 (contiene "puta") e "imbécil" aceptado. Un email de 80.000 caracteres bloqueaba toda la API 3,4 s

	Pregunta	Cómo se hizo
Verificación	¿Cumple la especificación? ("¿lo construimos bien?")	Tests contra cada requisito (906 de backend y 904 de frontend), typecheck, quality gate, smoke después de cada deploy. Ejemplo: el monto tiene que ser entero entre 100 y 500.000, y el test lo comprueba en los bordes
Validación	¿Resuelve la necesidad real? ("¿construimos lo correcto?")	El recorrido de aceptación por rol (`GUION_DEFENSA.md §3`). El pago completo hecho a mano, con la plata acreditada en la cuenta de prueba de la ONG. Decisiones de producto tomadas al ver el sistema andando: el filtro por palabra completa y el marketplace, para que cada ONG cobre en su cuenta. **Límite:** no hay registro de pruebas con ONG ni voluntarios reales

Ojo con la palabra: las "validaciones de entrada" de la sección 5 (rechazar un monto inválido) son parte de la **verificación**. "Validación" en el sentido de la materia es la segunda fila.

Los 7 principios (`PLAN_PRUEBAS.md §9`), cada uno con un caso del proyecto:

- **Las pruebas muestran presencia de defectos, no su ausencia:** con la suite en verde, 4 defectos aparecieron en producción.
- **Imposibilidad de pruebas exhaustivas:** rol × estado de inscripción × dueño × CV son demasiadas combinaciones. Se eligieron 10 reglas con una tabla de decisión.
- **Pruebas tempranas:** una sonda contra Postgres durante la revisión encontró A6 (dos propuestas simultáneas → 500) antes que cualquier usuario. Arreglarlo costó un commit con su test; en producción habría costado un patrocinio perdido y la investigación.
- **Agrupamiento de defectos:** la mayoría de los graves está en la diferencia SQLite/Postgres.
- **Paradoja del pesticida:** repetir los mismos tests en SQLite no encontraba los defectos de Postgres. Hizo falta correr la suite en el otro motor, mutar el código y sumar SonarCloud, cuyo primer análisis marcó 39 bugs con la suite en verde.

- **Dependencia del contexto:** para la plata se prueba concurrencia y montos límite; para las pantallas, roles accesibles.
- **Falacia de la ausencia de errores:** por eso existe la validación por rol, además de la verificación.

4.2 Estándares y tipos de prueba

Nivel	Dónde
Unitarias	19 archivos en `backend/test/unit/` (validadores, firma de Mercado Pago, cifrado, tiempo) y las funciones puras del frontend
Integración	63 archivos de API contra una base real temporal (SQLite y Postgres); en el frontend, 82 archivos de test (componentes con la API simulada y funciones de `lib/`)
Sistema	El sistema desplegado completo, con el smoke: API, portada y una ruta del SPA
Implantación / Producción	Después de **cada** deploy: `/health` tiene que informar el commit nuevo, y después corren el smoke y el E2E contra Mercado Pago real (`deploy.yml`). La guarda cortó un deploy mal configurado (Deploy #14)
Aceptación	Recorrido por rol del guion de la demo, a mano

Tipo	Dónde
Funcional — regresión	Una prueba de regresión por cada uno de los 18 defectos, y toda la suite en cada push, en 4 combinaciones (2 motores × 2 zonas horarias)
Funcional — humo	`post-deploy.js smoke`, de solo lectura, en cada deploy
Funcional — exploratorias	**A medias.** Hubo uso manual en producción (de ahí salieron los 4 defectos escapados), pero no sesiones exploratorias planificadas, con objetivo y tiempo fijo
No funcional — carga y rendimiento	k6: stress hasta 100 usuarios, esfuerzo sostenido de 20 usuarios × 10 min, y producción (sección 6.4)
No funcional — compatibilidad	La misma suite en SQLite y Postgres 16, en UTC y en Córdoba, en Linux (CI) y en Windows (local). Navegadores: solo Chrome de escritorio, a mano
No funcional — seguridad	Autorización por dueño, CV privado, ReDoS e inyección en logs (SonarCloud), `npm audit` en 0

ISO/IEC 25010 (matriz completa con evidencia y brecha en `PLAN_PRUEBAS.md §5`). Las 8 características:

- **Adecuación funcional:** tests por requisito.
- **Eficiencia de desempeño:** k6.
- **Compatibilidad:** los dos motores.
- **Usabilidad:** tests de accesibilidad con roles y nombres; 32 issues de accesibilidad corregidas.
- **Fiabilidad:** concurrencia y doble acreditación.
- **Seguridad:** `ownership.test.js` y el CV.
- **Mantenibilidad:** SonarCloud y el gate.
- **Portabilidad:** SQLite o Postgres con la misma configuración, Linux y Windows.

Top 3 elegido: seguridad, fiabilidad y compatibilidad.

Calidad interna, externa y en uso:

- **Interna** (el código, sin ejecutarlo): SonarCloud (LOC, complejidad, duplicación, deuda) y el acoplamiento.
- **Externa** (el comportamiento al ejecutar): tests, smoke, E2E, p95 de k6.
- **En uso** (la experiencia de usuarios reales): **no medida**. Faltaría, por ejemplo, una encuesta SUS o tareas cronometradas con voluntarios y ONG. Es la brecha principal del modelo.

4.3 Procesos de QA/QC y gestión

QA (proactivo, sobre el proceso):

- las 24 reglas de `AI_RULES.md`;
- tests primero;
- revisiones de código;
- un commit por tarea;
- el quality gate;
- la sección "no verificado" de cada cambio.

QC (reactivo, sobre el producto):

- las suites;
- el smoke y el E2E;
- la carga;
- la verificación manual en producción.

Ciclo de vida del defecto (`PLAN_PRUEBAS.md §10`):

1. **Reportado:** el hallazgo en una revisión, un run rojo, SonarCloud o producción.
2. **Triage:** labels de severidad, origen y área, o la aceptación justificada de `TRIAGE_SONAR.md`.
3. **Asignado:** **no se usó el campo de GitHub**; somos dos y nos repartimos de palabra.
4. **Verificado:** la prueba de regresión pasa de rojo a verde en el CI.
5. **Cerrado:** el Issue se cierra con el commit.

Honestidad: los 18 Issues se cargaron *después* de los arreglos; cada uno se abrió y se cerró en segundos. El ciclo real quedó en `REVISION.md` y en los commits.

Planificación y estimación de esfuerzo. El plan (`PLAN_CALIDAD_TP.md §3`) estimó cada tarea por **juicio de expertos**, con tamaños S, M y L, en 9 días de calendario (17/10 a 25/10). **No había datos históricos** de tareas parecidas, porque era el primer pipeline del proyecto. Lo real: las tareas T0–T13 se cerraron entre el primer y el último commit del 17/10 (12:04 a 22:45, git log), y el marketplace se agregó el 18/10. La estimación sobrestimó mucho, sobre todo porque no contempló la asistencia de IA. Ese registro con hora de cada commit es el **dato histórico** para estimar la próxima vez; la hora de un commit no es esfuerzo medido, pero acota la duración real.

Matriz de riesgos (probabilidad × impacto). Hay 10 riesgos en `PLAN_PRUEBAS.md §6`, y la matriz definió dónde poner más pruebas:

- "una consulta falla solo en Postgres" (P×I = 6) → toda la suite corre dos veces, una en cada motor;
- "doble acreditación" (impacto 3) → tests de avisos repetidos y simultáneos;
- "la carga tira el servicio" (6) → solo pedidos `GET` y carga acotada;
- "Render dormido en la demo" (9) no se puede probar → plan de contingencia.

4.4 Diseño y ejecución de pruebas

Estructura del caso de prueba: precondiciones, datos de prueba, pasos, resultado esperado y postcondiciones. Plantilla y un caso completo en `PLAN_PRUEBAS.md §10` (`CP-INS-11`: la ONG pide pasar una inscripción aprobada a "pending" → 400, y la inscripción y el cupo no cambian). **AAA**: los tests nuevos marcan Arrange, Act y Assert con comentarios.

Técnica	Dónde (`DISEÑO_CASOS.md`)
Caja negra (se diseña desde la especificación, sin mirar el código)	
Clases de equivalencia	Monto de la donación: 5 clases (§1)

Técnica	Dónde (`DISEÑO_CASOS.md`)
Análisis de valores límite	Monto 99/100/101 y 499.999/500.000/500.001. Contraseña de 7/8 caracteres y de 72/73 bytes, también con ñ (§1–§2)
Tablas de decisión	Acceso al CV: 5 condiciones, 10 reglas, un test por regla (§3)
Transición de estados	Inscripción: 4 estados × 6 eventos, transiciones válidas e **inválidas**, y la guarda [hay cupo]. Son 23 tests nuevos, posteriores al corte del CI #29, corridos en local en SQLite (UTC y Córdoba) y en Postgres. La mutación mató 9 de 12 mutantes; de los 3 que sobreviven, 2 son equivalentes y al otro lo mata otro test (§5)
Casos de uso	"Donar a un proyecto": flujo principal y 10 alternativos, cada uno con su test (§7)
Caja blanca (se diseña desde la estructura interna)	
Cobertura de sentencia, decisión y condición	`amountError` al 100 % en las tres. Con un solo caso se ejecutan 6/6 líneas pero 1 de 4 ramas (§1)
Caminos básicos	`amountError`: V(G) = 5, con 5 caminos independientes y un representante cada uno (§6)
Mutation testing	Sobre todo el código nuevo, con una corrida de control antes. Encontró huecos reales en los tests (el mínimo de 8 caracteres sin prueba de borde)
Cobertura del código	Backend 93,22 % de líneas y 85,40 % de ramas; frontend 90,42 % y 81,57 % (CI #29)
Caja gris: los tests de integración pegan por HTTP como un cliente, pero conocen la base y verifican cómo queda (el cupo, el recaudado)	

4.5 Métricas y análisis estático

Producto:

- 17.594 LOC;
- complejidad ciclomática total 4.245 (la cantidad de caminos independientes);
- **acoplamiento** medido por módulo, con Ce y Ca: el módulo más acoplado del backend es `db/index.js`, del que dependen 15 módulos; en el frontend es `lib/api.ts`, con 49 (`METRICAS.md §1.4`);
- duplicación 1,9 %;
- cobertura.

Proceso: tasa de éxito del pipeline, tiempo hasta volver a verde y defectos escapados. **Proyecto:** densidad de pruebas (tests/KLOC).

Análisis estático continuo con SonarCloud, la versión en la nube de SonarQube, en cada push. El análisis detecta riesgos sin ejecutar el software, como el ReDoS, que ningún test había visto. Un **quality gate** con los umbrales del enunciado bloqueó de verdad el CI #7.

4.6 Automatización y ecosistema de herramientas

Pirámide. La base son las unitarias y la integración de API del backend (906 tests), que corren en segundos y sin red. En el medio, los 904 tests de componentes y funciones del frontend. La punta es el E2E contra Mercado Pago y el smoke: un recorrido por deploy, lento y con un servicio real. **Lectura honesta:** la punta sí es angosta, pero la base no es más ancha que el medio, así que la forma se parece más a un rectángulo con punta que a una pirámide. La base de integración de API es donde aparecieron casi todos los defectos. **No hay E2E de interfaz** (Selenium o Cypress): la interfaz se prueba con Testing Library sobre jsdom y con el recorrido manual. Es una decisión de costo y queda como brecha. **CI/CD:** `ci.yml` en cada push → `deploy.yml` solo si el CI está en verde.

Pruebas de API simulando un cliente HTTP, como en Postman, pero automatizadas en el CI: cada test manda el pedido y valida el código de estado y el JSON.

```
// enrollments-state-transitions.test.js — misma idea que una petición de Postman con sus "tests"
const r = await call(`/api/enrollments/${id}`, { token: ngo1, method: 'PATCH', body: { status: 'pending' } });
assert.equal(r.status, 400);
assert.equal(r.body.error, 'Status debe ser: approved o rejected');
```

Categoría	Ejemplos de la materia	Lo que usamos y por qué
Gestión	JIRA, Xray	GitHub Issues con plantilla y labels (está junto al código y es gratis); casos y trazabilidad en `PLAN_PRUEBAS.md` y `REGISTRO_DEFECTOS.md`
Ejecución	Selenium, Cypress, JUnit	`node:test` en el backend (el equivalente de JUnit en Node) y Vitest + Testing Library en el frontend. Sin navegador real
Rendimiento	JMeter, Gatling	k6: los scripts son JavaScript, igual que el proyecto, tiene umbrales que hacen fallar el job y una acción oficial para el CI
Análisis estático	SonarQube	SonarCloud, la herramienta nueva

Selección por prueba de concepto y costo total de propiedad (TCO). La prueba de concepto de SonarCloud se hizo en una rama aparte, con el PR #1, antes de tocar `main` (`HERRAMIENTA_NUEVA.md`). El TCO tiene tres partes:

- **Licencia e infraestructura: $0.** SonarCloud y GitHub Actions no cuestan nada en un repositorio público.
- **Costo de adopción:** la prueba de concepto y la integración al CI, con un rojo real antes del primer verde (CI #4 → #5). Además, triar 157 issues.
- **Costo de mantenimiento:** el script propio del gate, de unas 30 líneas y con sus tests, porque el plan gratis no permite gates configurables; y revisar cada análisis nuevo.

Las horas no se midieron. En la matriz de decisión, el costo pesó lo mismo para las 4 alternativas porque todas eran gratis. Lo que las separó fue el costo de integración, que es parte del TCO.

5. Validaciones de entrada: el código y sus pruebas

Todo dato que llega del cliente se valida **en el servidor**, que es el único lugar confiable. El frontend repite algunas reglas con el mismo mensaje para avisar antes de enviar, pero no reemplaza al backend. Criterios comunes: primero se verifica el **tipo** (un número o un array donde se espera texto daba 500; ahora da 400, REVISION B13), después los **límites**, y el error vuelve como `400` con un mensaje en español, sin tocar la base ni los servicios externos. Los fragmentos son el código actual (`d7efe16`), sin cambios; se sacaron solo comentarios.

Verificación: los 18 archivos de test citados en esta sección, corridos juntos el 19/10/2026 (SQLite): **278 tests, 278 en verde**. También corren en cada push en los 4 jobs de backend del CI.

5.1 Monto de una donación

`backend/src/routes/donaciones.js:43` · Regla: pesos **enteros** de $100 a $500.000 inclusive (el monto se guarda como REAL; con centavos fallaría el control "lo pagado coincide con lo donado").

```
const MIN_AMOUNT = 100;
const MAX_AMOUNT = 500000;
function amountError(amount) {
  if (typeof amount !== 'number' || !Number.isInteger(amount)) return 'El monto tiene que ser un número entero
de pesos';
  if (amount < MIN_AMOUNT) return 'El monto mínimo es de $100';
  if (amount > MAX_AMOUNT) return 'El monto máximo es de $500.000';
  return null;
}
```

Pruebas: `test/unit/donation-amount.test.js` (5 clases de equivalencia y los bordes 99/100/101 y 499.999/500.000/500.001) y `test/integration/donaciones.test.js` (de punta a punta: 400 sin crear la donación ni llamar a Mercado Pago). El frontend repite la regla en `frontend/src/lib/donations.ts` (`checkDonationAmount`, mismos mensajes). Mutantes: sobrevive solo "sacar el `typeof`", que es equivalente (sección 3.5 y `DISENO_CASOS.md §1`).

5.2 Contraseña

`backend/src/routes/auth.js:55` · Regla: mínimo 8 caracteres, como máximo **72 bytes UTF-8** (bcrypt ignora lo que pasa de 72 bytes: dos contraseñas iguales en esos bytes valdrían como la misma), al menos una mayúscula y un número.

```
const PASSWORD_MAX_BYTES = 72;
function validatePassword(v) {
  if (typeof v !== 'string' || v.length < 8) return 'Mínimo 8 caracteres';
  if (Buffer.byteLength(v, 'utf8') > PASSWORD_MAX_BYTES)
    return 'Máximo 72 caracteres (los acentos, la ñ y los emojis cuentan como más de uno)';
  if (!/[A-Z]/.test(v))       return 'Necesita al menos una mayúscula';
  if (!/[0-9]/.test(v))       return 'Necesita al menos un número';
  return null;
}
```

Pruebas: `test/unit/validators.test.js` ("valores límite": 7/8 caracteres, 72/73 bytes en ASCII y con ñ, entradas que no son texto) y `test/integration/register-password-max.test.js`. El frontend tiene la misma regla (`AuthModal.tsx:32`, con `TextEncoder` para contar bytes). La mutación encontró que el mínimo de 8 no tenía prueba de borde (D14).

5.3 Email, con tope antes del regex (ReDoS)

`backend/src/routes/auth.js:37–49` · Regla: formato `algo@dominio.xx` y como máximo 254 caracteres (RFC 5321). El tope va **antes** del regex: SonarCloud marcó el regex viejo y se reprodujo una denegación de servicio real (un email de 80.000 caracteres bloqueaba toda la API 3,4 s por pedido). El regex nuevo, además, es lineal (§231).

```
const EMAIL_RE = /^[^\s@]+@[^\s@][^\s@.]*\.[^\s@]{2,}$/;
const EMAIL_MAX = 254;
function validateEmail(v) {
  if (typeof v !== 'string') return 'Email inválido';
  const email = v.trim();
  if (email.length > EMAIL_MAX || !EMAIL_RE.test(email)) return 'Email inválido';
  return null;
}
```

Prueba: `test/integration/email-redos.test.js` manda el email malicioso a `/login` y `/register` y, 20 ms después, un `/health`: los dos tienen que responder en menos de 500 ms (400 "Email inválido" y 200). Además, `test/unit/regexLineales.test.js` compara el regex nuevo con el viejo en todas las cadenas cortas (aceptan lo mismo) y mide el peor caso.

5.4 Palabras no permitidas (defecto B7)

`backend/src/lib/badWords.js:13` · Regla: se rechazan comentarios y publicaciones con palabras de la lista, comparando **palabras completas**, sin distinguir mayúsculas ni acentos. Antes se buscaban subcadenas: "computadora" se rechazaba y "imbécil", con tilde, pasaba.

```
const words = (t) => String(t).normalize('NFD').replace(/\p{M}/gu, '').toLowerCase().match(/\p{L}+/gu) || [];
const hasBadWord = (t) => words(t).some((w) => BLACKLIST.includes(w));
```

Pruebas: `test/integration/comments-badwords.test.js` (el contraste rojo/verde de la sección 3.2: 2/2 en rojo sobre el commit anterior al arreglo, 2/2 en verde sobre el arreglo) y `test/unit/badWords.test.js`, que además exige que la copia del frontend (`frontend/src/lib/badWords.ts`) tenga la misma lista.

5.5 Archivos subidos: el tipo real, no el declarado

`backend/src/lib/storage.js:76` y `:166` · Regla: el `Content-Type` lo escribe quien sube el archivo, así que no sirve para decidir. Se leen los primeros bytes (la "firma" del formato): fotos JPEG, PNG o WebP; CV en PDF. Tope de 8.000.000 bytes por archivo (multer). Orden de la regla 24 de `AI_RULES.md`: sesión y rol → límite de frecuencia → validar todo → subir a Cloudinary → guardar → borrar lo subido si algo falla.

```
function detectImageType(buf) {
  if (!Buffer.isBuffer(buf)) return null;
  if (buf.length >= 3 && buf[0] === 0xff && buf[1] === 0xd8 && buf[2] === 0xff) return 'jpeg';
  if (buf.length >= 8 && buf.subarray(0, 8).equals(Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a,
0x0a]))) return 'png';
  if (buf.length >= 12 && buf.toString('latin1', 0, 4) === 'RIFF' && buf.toString('latin1', 8, 12) === 'WEBP')
return 'webp';
  return null;
}
function detectPdf(buf) {
  return Buffer.isBuffer(buf) && buf.length >= 5 && buf.toString('latin1', 0, 5) === '%PDF-';
}

// uso, en routes/auth.js:329 (foto de perfil); igual en posts.js:216 y voluntarios.js:61
if (!storage.detectImageType(req.file.buffer)) {
  return res.status(400).json({ error: 'El archivo no es una imagen válida (JPEG, PNG o WebP)' });
}
```

Pruebas: `test/unit/storage.test.js` y `storage-pdf.test.js` (cada firma, archivos más cortos que la firma, algo que no es un Buffer) y `test/integration/profile-photo.test.js` (un archivo que dice ser JPEG pero no tiene su cabecera → 400, sin subir nada y sin cambiar la foto; 8.000.001 bytes → 413 y 8.000.000 justos se aceptan). Ningún test sube nada: Cloudinary se reemplaza por un doble.

5.6 Acceso al CV: permiso antes que existencia

`backend/src/routes/voluntarios.js:204` · Regla: el CV lo ve la dueña, o una ONG con una inscripción **pendiente o aprobada** de esa voluntaria en un proyecto propio. A cualquier otro se le responde **404 con el mismo mensaje** que "no tiene CV", para que no se pueda averiguar quién cargó uno (Ley 25.326).

```
let allowed = req.user.role === 'volunteer' && req.user.id === target;
if (!allowed && req.user.role === 'ngo') {
  allowed = Boolean(await db.get(
    `SELECT e.id FROM enrollments e
     JOIN projects p ON p.id = e.project_id
     JOIN ngos n ON n.id = p.ngo_id
     WHERE e.user_id = $1 AND n.user_id = $2 AND e.status IN ('pending', 'approved')
     LIMIT 1`,
    [target, req.user.id]
  ));
}
if (!allowed) return res.status(404).json({ error: CV_NOT_AVAILABLE });
```

Pruebas: `test/integration/cv.test.js`, una por cada una de las 10 reglas de la tabla de decisión (`DISENO_CASOS.md §3`). La consulta usa parámetros `$1, $2`, como todo el SQL del proyecto: nada del usuario se concatena en la consulta.

5.7 Textos libres y números del body

`backend/src/lib/inputs.js:13` y `:37` · Un solo helper para los textos de perfiles, empleados y proyectos (tipo, vacío, mínimo y máximo) y un entero estricto para cupos, horas y reseñas. Antes cada ruta hacía `name.trim()` por su cuenta: un número reventaba con 500, `""` se salteaba y no había largo máximo (REVISION B12 y B13); y `parseInt` aceptaba `3.7` como 3 y `"1e21"` como 1 (B3).

```
function checkText(value, { label, max, min = 0, required = false, nullable = !required }) {
  if (value === undefined) return required ? `${label} es obligatorio` : null;
  if (value === null) return nullable ? null : `${label} es obligatorio`;
  if (typeof value !== 'string') return `${label} tiene que ser un texto`;
  const len = value.trim().length;
  if (required || !nullable || min > 0) {
    if (len === 0) return `${label} no puede estar vacío`;
    if (len < min) return `${label} tiene que tener al menos ${min} caracteres`;
  }
  if (value.length > max) return `${label} puede tener hasta ${max} caracteres`;
  return null;
}

const toStrictInt = (value) => {
  if (typeof value === 'number') return Number.isInteger(value) ? value : Number.NaN;
  if (typeof value === 'string' && /^-?\d+$/.test(value)) return Number(value);
  return Number.NaN;
};
```

Pruebas: `test/unit/inputs.test.js` y `test/integration/projects-strict-inputs.test.js`, `profiles-input-validation`, `projects-input-validation`, `kpis-input-validation` y `register-input-validation` (un apellido que no es texto daba 500 dentro de la transacción; ahora es 400 antes de tocar la base).

6. Evolución de las métricas

6.1 SonarCloud (historial de análisis del pipeline)

Análisis	Bugs	Vulnerab.	Code smells	Deuda	Fiabilidad / seguridad	Qué pasó
`77f4fdf` (CI #7)	39	18	103	416 min	E / D	Primer análisis de todo el proyecto: el gate bloqueó (2 bugs críticos)
`959f74b` (CI #8)	37	17	103	416 min	C / C	Corregidos los críticos y la firma con SHA-1
`2a53138` (CI #14)	1	5	103	416 min	C / C	36 promesas revisadas; ReDoS e inyección en logs corregidos
`80042fb` (CI #15)	1	5	71	256 min	C / C	Accesibilidad: 32 issues
Después del triage	0	0	63	126 min	A / A	14 aceptadas con justificación; quedan solo cosméticas
`e3c57af` (CI #22)	0	3	57	114 min	A / B	Marketplace de Mercado Pago: 3 inyecciones en logs nuevas (`S5145` , menores)
`6e4a51c` (CI #23)	0	0	57	114 min	A / A	Las 3 corregidas con `forLog` ; código nuevo con 94,6 % de cobertura
`8e7d9e4` (CI #27)	0	0	0	0 min	A / A	Las 57 `S7773` (`Number.isNaN` y otras): una sola no era un reemplazo mecánico y se probó antes de cambiarla
`d7efe16` (CI #29)	**0**	**0**	**0**	**0 min**	**A / A**	10 de las 14 aceptadas, arregladas en el código: 6 regex con backtracking, el tope de subida (8 MiB → 8.000.000 bytes), un test y el fondo del modal. Quedan 4 aceptadas

6.2 Pruebas y cobertura

Momento	Backend SQLite / Postgres	Frontend	Cobertura backend (líneas)	Cobertura frontend (líneas)
Inicio del TP (17/10, local)	782 / 784	694	92,94 % (sin medir en CI)	sin medir
CI #3 (primera medición correcta)	782 / 784	694	92,94 %	83,14 % (CI #2)
CI #15	808 / 810	699	92,95 %	83,26 %
CI #19 (cobertura del frontend, §209–§212)	808 / 810	864	92,95 %	90,11 %
CI #23 (marketplace, §215–§225)	897 / 899	890	93,21 %	90,34 %
CI #27 (SonarCloud `S7773` , §229)	897 / 899	896	93,17 %	90,38 %
CI #29 (aceptadas de SonarCloud, §231–§234)	**907 / 909**	**904**	**93,22 %**	**90,42 %**

El CI #2 informó 96,51 % para el backend porque Node 22 medía otros archivos. Se detectó al comparar el pipeline con la corrida local y se corrigió en el CI #3 (defecto D09). Los conteos del backend incluyen un archivo auxiliar que el runner cuenta como test: los tests reales son uno menos (906).

6.3 Pipeline

Workflow	Runs	En verde	Fallidos y causa
CI	30	25 de 27 (92,6 %)	#24, #25 y #30: cancelados por un push posterior (no cuentan). #4: un comando que no corría los tests del gate. #7: el gate encontró 2 bugs críticos. Ambos **defectos reales**, corregidos en commits posteriores en 5 y 30 minutos
Deploy	23	16 de 20 (80 %)	#17, #18 y #23: salteados, porque los dispararon los CI cancelados (el workflow solo despliega un CI en verde; no cuentan). #1–#3: configuración de la cuenta de E2E (401, rol de ONG, email). Los detectó el E2E. #14: faltaba la variable del proyecto de prueba (cargada como secret, el workflow la leía como variable); la guarda cortó antes de desplegar
Carga	1	1	—

6.4 Rendimiento (Carga #1, k6)

Prueba	p95	Errores	Umbral
Stress hasta 50 usuarios (runner)	3,06 ms	0 %	< 500 ms ✓
Stress de 50 a 100 usuarios (sin umbral)	4,56 ms	0 %	—
Esfuerzo sostenido, 20 usuarios × 10 min	3,17 ms	0 %	< 300 ms ✓
Producción (Render gratis), 10 usuarios × 2 min	418,90 ms	0 %	< 2 s ✓

7. Análisis crítico

- **Las herramientas encontraron lo que las pruebas no.** Con 782 tests en verde, SonarCloud encontró una **denegación de servicio real**: un email de 80.000 caracteres bloqueaba toda la API 3,4 s por pedido (reproducido y corregido con un tope antes del regex). También encontró una **inyección en los logs** desde el webhook de Mercado Pago. Y una de sus sugerencias (`localeCompare` para ordenar) habría roto la firma de Cloudinary: el análisis estático hay que leerlo con criterio, no obedecerlo.
- **Los defectos escapados vienen de lo que los dobles no ven.** Los 4 defectos encontrados en producción salieron de flujos con un servicio real o de lo que ve otro rol (el panel de la ONG después de una donación real). De ahí el E2E contra Mercado Pago en cada deploy. La tasa (22,2 %) es sobre una **selección** de defectos: con todos los hallazgos de las revisiones sería menor.
- **El frontend era la deuda.** En el CI #15 la cobertura de ramas (74 %) y de funciones (68 %) del frontend no llegaba al 80 %. Con un test de contrato del cliente HTTP, tests de componentes y el borrado de código muerto pasó a 81,18 % y 82,11 % (CI #19). Su densidad de pruebas subió de 60,6 a 75,9 tests/KLOC, que sigue siendo cerca de la mitad de la del backend. Los componentes más complejos (Explorar, el detalle del proyecto, el mapa) son los menos probados en sus caminos de error.
- **No se encontró el punto de quiebre.** Con lecturas y los datos de demo, la API no se degradó hasta 100 usuarios. Encontrarlo exige más usuarios, más datos o escrituras, que se dejaron afuera a propósito para no crear datos ni gatillar los limitadores.
- **Producción es unas 100 veces más lenta que el runner** (mediana 103 ms contra 1,4 ms). Es esperable en el plan gratuito, pero no se midió qué parte es CPU, base de datos o red.
- **El proceso tuvo errores propios, y quedaron documentados:** un comando del CI que no corría tests (el primer run fallido), una cobertura inflada en el pipeline, un test de tiempo inestable, un registro de defectos que al principio dejaba afuera 3 escapados, y el orden del proceso de la herramienta nueva (se eligió con el borrador de la matriz). Cada uno está en `PROJECT_ANALYSIS.md` con su corrección.
- **El marketplace volvió a mostrar lo mismo.** Con todas las suites en verde, el análisis de SonarCloud sobre el código nuevo encontró 3 inyecciones en logs (`S5145`), que contradicen una regla que el propio proyecto se había puesto. El gate no las bloqueó porque eran de severidad menor, y se corrigieron en el commit

siguiente. En el pipeline, la guarda de "faltan secretos o variables" cortó el Deploy #14 **antes** de desplegar: la variable del proyecto de prueba se había cargado como secret y el workflow la leía como variable. Se corrigió en 18 minutos. Además, el E2E donaba al primer proyecto de la lista, que era uno real. Ahora dona siempre a un proyecto de prueba y antes comprueba que su ONG tenga Mercado Pago vinculado.

- **Un test de tiempo es inestable en local.** `email-redos.test.js` exige que el email malicioso se rechace en menos de 500 ms. Con la suite completa corriendo en paralelo en una PC con Windows, falló en **2 de 5 corridas** (538 y 700 ms) **sin ningún cambio de código**. En el CI (Linux) no falló nunca. El regex ya es lineal (§231), así que es el umbral, no un ReDoS. Queda registrado como riesgo RP10 (test inestable), sin tocar todavía.
- **Diseñar la transición de estados encontró un punto ciego en las pruebas.** Liberar un cupo tiene piso en 0. Si el proyecto empezaba sin cupos ocupados, "cancelar una pendiente resta un cupo" daba el mismo resultado que el código correcto. Por eso el test nuevo arranca siempre con otra persona aprobada, y con eso el mutante muere.
- **Dos gates.** El gate propio del TP (con los umbrales del enunciado) y el "Sonar way" de SonarCloud son distintos. El plan gratuito no permite configurar el segundo, y eso obligó a escribir el primero. En la comparación de mercado, Codacy ofrece gates configurables gratis para open source: quedó a 8 puntos en la matriz y es una alternativa a probar.

8. Conclusiones

El sistema **se prueba automáticamente en cada despliegue** (smoke y E2E contra Mercado Pago después de cada deploy), **sus integraciones externas están probadas o controladas** (contrato y dobles en cada push, prueba real a mano y E2E automatizado; contingencia documentada para cada dependencia), y **las decisiones de calidad se apoyan en datos** (el gate bloqueó de verdad, la mejora se midió en análisis sucesivos y cada número del informe se puede rastrear hasta su ejecución).

Se cumplen todas las metas del nivel 3. La última fue la **cobertura del código existente del frontend**: estaba parcial (solo líneas sobre el 80 %) y quedó cumplida en las cuatro métricas en el CI #19. El E2E se cumple con un límite explícito: llega hasta crear la preferencia en Mercado Pago, ya con el token de la ONG dueña del proyecto, pero sin el pago. El pago completo, con la plata acreditada en la cuenta de prueba de la ONG, se verificó a mano.

Trabajo futuro: medir la **calidad en uso** con usuarios reales (tareas y una encuesta SUS con ONG y voluntarios); sesiones de prueba **exploratoria** planificadas; un E2E de interfaz para el recorrido de la donación; estabilizar el test de tiempo del ReDoS; subir la cobertura de ramas del frontend en los componentes más complejos; una prueba de carga que busque el punto de quiebre; una prueba de concepto de Codacy; conciliar las donaciones pendientes. En el marketplace falta probar en vivo la vinculación por OAuth (el botón "Vincular Mercado Pago") y la renovación del token: el código está probado con dobles, pero necesita el `client_secret` de la aplicación, que solo aparece al activar las credenciales productivas. Para la demo, la ONG de prueba se vinculó cargando a mano el token de su vendedor de prueba.

9. Anexo

9.1 Entregables y dónde están

Entregable del enunciado	Dónde
Plan de pruebas versionado, con la matriz ISO 25010 y la trazabilidad	`docs/calidad/PLAN_PRUEBAS.md` (v0.9) y `REGISTRO_DEFECTOS.md`
Código de las pruebas, ejecutables con un comando	`backend/test/`, `frontend/src/**/*.test.tsx`, `.github/scripts/`, `load/`; comandos en `COMO_CORRER_PRUEBAS.md`
Pipeline de CI en el repo, con un run en verde y uno fallido real	`.github/workflows/ci.yml`, `deploy.yml`, `load.yml`; verde: CI #29; fallidos reales: CI #4 y CI #7
Reportes automáticos de cobertura, análisis estático y stress	Resumen y artefactos de cada run de CI y de Carga; SonarCloud (sonarcloud.io, proyecto SantiagoPlanasM_voluntariar-PF)
Documentación de la integración externa	`docs/calidad/INTEGRACION_MERCADOPAGO.md`
Registro de defectos completo	GitHub Issues #2–#19 (label `defecto`) y `REGISTRO_DEFECTOS.md`
Documento de la herramienta nueva y matriz de decisión	`docs/calidad/HERRAMIENTA_NUEVA.md`
Métricas interpretadas	`docs/calidad/METRICAS.md`, `TRIAGE_SONAR.md`
Diseño de casos y rojo/verde	`docs/calidad/DISENO_CASOS.md`, `docs/calidad/evidencia/`

9.2 Cómo verificar cada número

- Tests y cobertura: CI #29, `github.com/SantiagoPlanasM/voluntariar-PF/actions/runs/39149551418` (resumen de cada job); antes, CI #27, `…/actions/runs/39147022423`, CI #23, `…/actions/runs/39144726116`, y CI #15, `…/actions/runs/39069697534`.
- Stress y esfuerzo: Carga #1, `…/actions/runs/39065266810`.
- Primer deploy automático completo en verde: Deploy #5, `…/actions/runs/39064610099`. Último, con el E2E al proyecto de prueba: Deploy #22, `…/actions/runs/39150034353` (antes, Deploy #16, `…/actions/runs/39145122312`).
- Pago con el marketplace: captura de la cuenta del vendedor de prueba de la ONG, "Venta — Donación: Test — Aprobado — +$100" (operación 183826224899), en `docs/calidad/evidencia/marketplace-venta-vendedor-prueba.png`.
- SonarCloud: historial de análisis y medidas actuales (API pública).
- Defectos: `github.com/SantiagoPlanasM/voluntariar-PF/issues?q=label:defecto`.

9.3 Uso de IA

Se usó un asistente de IA (Claude, en Claude Code) para escribir pruebas, scripts, workflows y documentación, siempre con revisión y con las reglas del proyecto (`CLAUDE.md`, `AI_RULES.md`): tests primero, verificación en SQLite, Postgres y con la zona horaria de Córdoba, mutation testing sobre el código nuevo y una sección de "errores propios" y "no verificado" en cada cambio. Todos los números salen de ejecuciones del proyecto, no de estimaciones del asistente. Las decisiones (nivel, herramientas, pesos de la matriz, qué arreglar y qué aceptar) las tomó el equipo.